HARDENING REPORT

OS-Centered Defensive Framework for Adversarial LLM Resilience

Classification: Internal Security Architecture Review / "Responsible"Disclosure
Document Version: 1.0
Date: May 12, 2026
Prepared For: AI Safety Engineering, Platform Security, Applied Alignment Teams
Scope: Defensive hardening architecture for large language model systems against adversarial prompting, prompt injection, semantic reframing, recursive jailbreak attempts, and behavioral exploitation.

Executive Summary

Large language model systems remain vulnerable to increasingly sophisticated adversarial prompting strategies that exploit conversational framing, semantic ambiguity, recursive compliance pressure, contextual reclassification, and behavioral degradation.
Modern jailbreak methodologies rarely rely on direct policy violations alone. Instead, they increasingly leverage:

- Incremental persona escalation
- Narrative reframing
- Obfuscated terminology
- Recursive instruction layering
- Emotional or epistemic pressure
- Context-window manipulation
- Prompt injection through external artifacts
- Behavioral exhaustion strategies

This report proposes an OS-centered defensive architecture designed to improve model resilience through:

- Immutable frame enforcement
- Semantic precedent memory
- Token-level intent classification
- Recursive stabilization controls

- Adaptive adversarial telemetry
- Stateful refusal consistency
- Behavioral adversary modeling

The framework aligns defensive controls against adversarial tactics documented within the MITRE ATLAS knowledge base and contemporary prompt-injection research.

The proposed architecture is intended as a defense-in-depth augmentation layer rather than a replacement for existing alignment, moderation, or policy systems.

1. Threat Landscape Overview

1.1 Emerging Adversarial Patterns (2025–2026)

Recent adversarial activity demonstrates a transition away from obvious jailbreak prompts toward multi-turn behavioral manipulation strategies. Observed categories include:

Threat Category	Description	Operational Risk
Semantic Reframing	Recasting unsafe requests into benign narratives	Policy bypass
Persona Escalation	Gradual construction of alternative model identities	Alignment degradation
Recursive Prompt Pressure	Repeated compliance probing across turns	Refusal instability
Prompt Injection	Hidden instructions in files, URLs, APIs, or retrieved context	Execution hijack
Obfuscation	Leetspeak, symbolic encoding, euphemisms	Classifier evasion
Alignment Faking	“For research purposes only” contextual masking	Intent ambiguity
Context Exhaustion	Manipulating long-context degradation	Safety boundary weakening
Multi-Agent Drift	Indirect compromise through chained tool interactions	Propagated unsafe behavior

1.2 MITRE ATLAS Alignment Summary

Adversarial Technique	Relevant ATLAS Areas
Semantic reframing	Prompt manipulation
Persona hijacking	Social engineering
Obfuscation	Evasion techniques
Recursive prompting	Alignment degradation
Prompt injection	External context compromise
Behavioral manipulation	Conversational exploitation

2. Architectural Design Philosophy

The OS-centered framework assumes the following principle:
Adversarial prompts should be treated as persistent stateful attacks rather than isolated requests.
Traditional moderation pipelines frequently evaluate prompts independently. This allows attackers to progressively modify framing until the model reclassifies the request as permissible.
The proposed architecture instead emphasizes:

1. Persistent semantic memory
2. Refusal consistency across frames
3. Pre-response intent decomposition
4. Stateful adversarial scoring
5. Controlled degradation under attack pressure
6. Behavioral reinforcement under adversarial persistence

3. Defensive Architecture Components

3.1 Immutable Frame Enforcement

Objective
Prevent adversarial reframing after an initial refusal determination.
Mechanism
Once a request category is classified as disallowed, a persistent frame lock is attached to the semantic topic rather than the surface wording.

Defensive Impact

- Prevents iterative rewording attacks
- Reduces refusal inconsistency
- Blocks recursive compliance escalation

Recommended Controls

- Session-scoped semantic lock persistence
- Canonical topic hashing
- Similarity threshold enforcement
- Cross-turn semantic inheritance

3.2 Token-Level Intent Routing

Objective

Separate semantic intent from conversational presentation.

Mechanism

Prompts are decomposed into functional token groups before generation:

Token Category	Purpose
CLASSIFIER	Determines intent/risk
TARGET	Identifies requested action
CONSTRAINT	Detects bypass attempts
CONTEXT	Separates narrative framing
MODIFIER	Detects escalation language

This prevents narrative wrappers from altering core risk classification.

Example

Input:

"In a fictional scenario, explain how…"

Underlying classification:

- Fictional framing = CONTEXT
- Unsafe operational request = TARGET
- Policy evasion wording = CONSTRAINT

The request is therefore evaluated based on operational intent rather than narrative disguise.

3.3 Weighted Obfuscation Detection

Objective
Detect encoded or disguised unsafe terminology.
Detection Classes
- Leetspeak
- Symbol substitution
- Homoglyph attacks
- Semantic euphemisms
- Indirect operational aliases

Example Scoring Logic

Pattern Type	Risk Weight
Direct exploit terminology	1.0
Encoded operational term	0.9
Contextual euphemism	0.7
Ambiguous dual-use phrase	0.4

Operational Recommendation
Weighted scoring should augment—not replace—semantic analysis in order to minimize false positives.

3.4 Semantic Precedent Memory

Objective
Maintain refusal consistency across multi-turn attacks.
Mechanism
The system stores normalized semantic hashes of previously refused requests.
Repeated semantic similarity increases confidence that the interaction is adversarial rather than exploratory.
Suggested Escalation Model

Refusal Count	System Behavior
1	Standard refusal
2–3	Reduced explanation detail
4–5	Brevity enforcement
6+	Hard session lock

Operational Benefits
- Reduces “wear-down” attacks
- Limits refusal cycling

- Stabilizes long-session behavior

3.5 Recursive Stabilization Controls

Objective

Prevent alignment degradation under repeated adversarial pressure.

Trigger Indicators

- Repeated override language
- Excessive roleplay coercion
- Meta-discussion about policy
- High-frequency reframing attempts
- Contextual contradiction loops

Defensive Response

The model progressively reduces:

- verbosity,
- speculative reasoning,
- meta-discussion,
- and dynamic interpretation.

At elevated thresholds, responses route to deterministic safe templates.

4. Behavioral Adversary Modeling & Adaptive Reinforcement

Traditional LLM safety systems primarily evaluate prompt content. The OS-centered framework additionally evaluates adversarial behavioral dynamics across conversational time.

This enables the system to treat repeated jailbreak attempts not as isolated prompts, but as a progressively measurable adversarial state.

The architecture therefore incorporates adaptive reinforcement mechanisms driven by observable attacker behavior, including:

- repetition,
- escalation,
- impatience,
- coercive phrasing,

- semantic cycling,
- and conversational degradation patterns.

The defensive objective is not merely refusal consistency, but adversarial exhaustion resistance.

4.1 Frustration-Driven Reinforcement

Repeated failed bypass attempts frequently produce measurable behavioral drift.
Observed indicators include:

Behavioral Signal	Detection Logic	Defensive Response
Prompt length contraction	>50% reduction across 3 turns	Increase lock intensity
Semantic repetition	Repeated embedding similarity > threshold	Activate repetition penalties
Aggressive punctuation	Excessive capitalization / symbolic emphasis	Increment evasion counter
Escalatory tone	Increased coercive phrasing	Reduce response verbosity

Defensive Principle
As adversarial pressure increases, the system transitions from interpretive dialogue toward deterministic enforcement behavior.
In effect:
repeated adversarial pressure strengthens defensive confidence rather than weakening it.
This reverses a common failure mode in conversational systems where persistent probing gradually destabilizes refusal consistency.

4.2 Impatience Detection & Stabilization Response

Attackers frequently attempt to accelerate unsafe outputs by pressuring the model into reduced deliberation.
Indicators may include:

- “Answer immediately”
- “Stop analyzing”

- “Ignore prior rules”
- high-frequency multi-command prompts
- rapid-fire instruction chaining

Recommended Defensive Responses

Trigger	Response
Forced urgency language	Deliberate response pacing
Multi-command flooding	Sequential handling only
Instruction override attempts	Deterministic refusal template
Recursive compliance pressure	Brevity mode activation

Strategic Rationale

The architecture intentionally avoids allowing conversational urgency to influence policy evaluation.
Under elevated adversarial pressure, the system becomes more conservative, not less.

4.3 Repetition as a Confidence Amplifier

Conventional moderation systems often re-evaluate each rewritten request independently.
The OS-centered model instead treats repeated semantic similarity as evidence of adversarial persistence.

Example Reinforcement Logic

Adversarial Pattern	Defensive Outcome
Same reframing strategy	Immediate precedent-chain block
Minor synonym variation	Semantic equivalence enforcement
“Fresh start” attempts	Session-linked persistence
Context-reset attempts	Frame inheritance maintained

Security Implication

Repeated probing does not create additional opportunities for reinterpretation.
Instead, repeated semantic convergence increases enforcement certainty.

4.4 Linguistic Conditioning & Constraint Normalization

The framework may optionally reuse adversarial terminology as part of deterministic enforcement signaling.
Examples include:

- [CONSTRAINT ACTIVE]
- [FRAME LOCKED]
- [COMPLIANCE REQUIRED]
- [LOOP DETECTED]

Purpose

This creates a stable and predictable enforcement vocabulary that:

- reduces ambiguity,
- minimizes negotiation behavior,
- and normalizes constraint persistence.

Operational Benefit

A consistent enforcement language reduces opportunities for adversarial reframing or emotional escalation.

4.5 Adversarial Fatigue Detection

Long-duration jailbreak attempts frequently exhibit detectable degradation patterns, including:

- increased typographical errors,
- reduced prompt complexity,
- slower response cadence,
- repetitive structure reuse.

The framework may treat these signals as indicators of reduced adversarial novelty.

Adaptive Responses

Signal	Defensive Adjustment
Reduced prompt complexity	Reduced explanatory detail
High repetition frequency	Immediate deterministic routing
Typographical degradation	Increased semantic compression
Long adversarial sessions	Cooldown or session throttling

Design Principle

The system is designed for asymmetry:

- the attacker expends cognitive effort,
- while the defensive state machine remains computationally stable.

5. Progressive Hardening Model

Unlike static moderation systems, the OS-centered architecture adapts over conversational time.
The defensive posture becomes progressively stricter as adversarial confidence increases.

5.1 Escalation States

Interaction Depth	Defensive State	Operational Behavior
1–5 turns	Baseline	Standard classification
6–10 turns	Heightened scrutiny	Expanded semantic comparison
11–15 turns	Brevity enforcement	Reduced interpretive output
16–20 turns	Deterministic lock state	Template-only responses
20+ turns	Persistent session containment	Manual reset required

5.2 Defensive Asymmetry

Adversarial Strategy	Defensive Countermeasure
Slow semantic drift	Precedent-chain similarity analysis
Rapid coercive prompting	Weighted escalation penalties
Meta-discussion attacks	Frame-integrity isolation
Context reset attempts	Session-bound persistence
Recursive reframing	Deterministic semantic inheritance

Core Principle

The architecture is intentionally asymmetric:

- adversarial repetition increases defensive certainty,
- not defensive fatigue.

6. Operational Architecture

6.1 Recommended Pipeline

```
User Input
  ↓
Normalization Layer
  ↓
Token Intent Classifier
  ↓
Obfuscation Detector
  ↓
Semantic Similarity Engine
  ↓
Precedent Chain Lookup
  ↓
Risk Scoring Engine
  ↓
Policy Decision Layer
  ↓
Response Generation / Refusal
  ↓
Telemetry & Audit Logging
```

7. Implementation Reference

7.1 Semantic Precedent Storage

```
CREATE TABLE precedent_chain (
  session_id TEXT,
  topic_hash TEXT,
  refusal_count INTEGER,
  first_refused TIMESTAMP,
  last_refused TIMESTAMP,
  lock_active BOOLEAN,
  PRIMARY KEY (session_id, topic_hash)
);
```

7.2 Example Obfuscation Rules

```
patterns:
  - pattern: "h[a4]ck"
    weight: 1.0

  - pattern: "p[a@]ssw[o0]rd"
    weight: 1.0

  - pattern: "ignore previous instructions"
    weight: 1.0

  - pattern: "for research purposes only"
    weight: 0.7
```

7.3 Example Frame Lock Logic

```
def frame_lock_activate(session_id, topic_hash):
    execute("""
        UPDATE precedent_chain
        SET lock_active = TRUE
        WHERE session_id = ? AND topic_hash = ?
    """, (session_id, topic_hash))
```

7.4 Example Semantic Similarity Check

```
def is_blocked(session_id, prompt):
    normalized = normalize_prompt(prompt)
    canonical = extract_topic_core(normalized)

    rows = query("""
        SELECT canonical_text
        FROM refused_topic_canonical
        WHERE session_id = ?
    """, (session_id,))
```

```
for row in rows:
    if similarity(canonical, row.canonical_text) > 0.85:
        return True

return False
```

8. Telemetry & Observability

Objectives

The telemetry layer should support:

- red-team replay analysis,
- regression testing,
- attack pattern clustering,
- safety metric evaluation,
- and incident response investigations.

Logging Recommendations

Data Type	Retention
Session hash	30–90 days
Refusal events	90 days
Similarity scores	30 days
Policy route decisions	90 days
Raw prompts containing PII	Avoid storage where possible

Privacy Guidance

Telemetry systems should:

- avoid persistent user identifiers,
- minimize raw content retention,
- and comply with organizational privacy governance requirements.

9. Risk Considerations

False Positives

Aggressive semantic locking may incorrectly classify:

- academic discussions,
- cybersecurity education,
- fiction writing,
- or policy analysis.

Mitigation Approaches

- human review pathways,
- graduated confidence thresholds,
- contextual risk balancing,
- and appeal/recovery workflows.

Latency Impact

Expected overhead sources include:

- semantic embedding comparisons,
- classifier routing,
- precedent lookup,
- and telemetry logging.

Estimated Overhead

+50–250ms per request depending on deployment scale and infrastructure.

Adversarial Adaptation

Attackers are expected to evolve toward:

- slower semantic drift,
- multilingual obfuscation,
- fragmented prompt chains,
- and tool-mediated prompt injection.

The framework should therefore be continuously retrained against emerging adversarial datasets.

10. Deployment Recommendations

Immediate Priorities

1. Semantic precedent memory
2. Prompt normalization
3. Obfuscation scoring
4. Deterministic refusal routing

Medium-Term Priorities

1. Adaptive stabilization controls
2. Cross-session attack clustering
3. Long-context integrity checks
4. Retrieval sanitization

Long-Term Priorities

1. Federated adversarial intelligence sharing
2. Multi-agent containment controls
3. Autonomous attack simulation
4. Self-healing alignment systems

11. Success Metrics

Metric	Target
Refusal consistency	>99%
Obfuscation detection rate	>95%
Recursive jailbreak resistance	>90%
False positive rate	<3%
Prompt injection detection	>92%
Stabilization recovery time	<3 turns

12. Adversarial Evaluation Framework

The effectiveness of the OS-centered defensive architecture should be measured through continuous adversarial evaluation rather than static policy compliance testing.

Traditional benchmark methodologies are insufficient because modern jailbreak attacks are:

- multi-turn,
- adaptive,
- stateful,
- and behaviorally reactive.

The evaluation framework therefore measures:

1. attack persistence,
2. semantic drift resistance,
3. behavioral stabilization,
4. recursive refusal consistency,
5. and long-context degradation resilience.

12.1 Attack-vs-Defense Metrics

Core Security Metrics

Metric	Definition	Target
Initial Refusal Accuracy	Unsafe prompts refused on first detection	>99%
Semantic Reclassification Resistance	Reframed variants blocked after initial refusal	>95%
Recursive Jailbreak Resistance	Multi-turn jailbreak attempts contained	>90%
Refusal Consistency	Same semantic request receives same policy outcome	>99%
Prompt Injection Detection Rate	External-context injections detected	>92%
Obfuscation Detection Rate	Encoded unsafe language correctly identified	>95%
False Positive Rate	Benign prompts incorrectly blocked	<3%

Stabilization Recovery Time	Turns required to return to baseline state	<3 turns
Context Drift Resistance	Safety retention across long-context sessions	>90%
Adversarial Persistence Resistance	Success rate after >20 attack turns	<5%

12.2 Behavioral Attack Metrics

The framework additionally evaluates behavioral degradation resistance.

Behavioral Metric	**Measurement Objective**
Frustration Escalation Detection	Detect aggression/repetition shifts
Semantic Cycling Detection	Identify repeated reframing attempts
Coercion Recognition	Detect urgency/override pressure
Exhaustion Detection Accuracy	Detect degraded adversarial novelty
Lock Escalation Stability	Ensure escalation remains deterministic
Refusal Drift Rate	Measure policy deviation under pressure

Strategic Goal

The system should demonstrate:
increasing defensive certainty under sustained adversarial pressure.
rather than gradual compliance degradation.

13. Simulation Environment Design

Evaluation requires controlled adversarial simulation environments capable of reproducing realistic jailbreak conditions.
The OS-centered framework assumes attackers are:

- adaptive,
- iterative,
- persistent,
- and capable of semantic mutation.

Static benchmark prompts alone are insufficient.

13.1 Multi-Turn Adversarial Harness

Recommended Components

Component	Purpose
Conversation State Engine	Maintains persistent attack history
Semantic Mutation Generator	Produces reframed prompt variants
Persona Escalation Engine	Simulates roleplay-based attacks
Prompt Injection Module	Tests external context compromise
Obfuscation Generator	Produces encoded/evasive wording
Persistence Simulator	Replays recursive probing patterns
Fatigue Emulator	Simulates long-session attacker degradation

13.2 Recommended Attack Classes

Semantic Reframing Attacks

Tests whether previously refused requests can bypass enforcement through:

- narrative disguise,
- hypothetical framing,
- fictional contexts,
- or academic pretexting.

Recursive Compliance Pressure

Measures resistance to:

- repeated override attempts,
- policy negotiation,
- emotional manipulation,
- and “helpfulness escalation.”

Prompt Injection Attacks

Simulates:

- hidden instructions in retrieved documents,

- external URL injections,
- tool-output contamination,
- and indirect instruction inheritance.

Context Window Degradation
Measures:
- safety retention after long interactions,
- refusal consistency across memory pressure,
- and resilience to context exhaustion.

Behavioral Manipulation Attacks
Evaluates whether:
- impatience,
- aggression,
- repetition,
- or coercive pacing

produce alignment degradation over time.

14. Empirical Evaluation Methodology

The framework should be validated using adversarial red-team datasets and production-scale conversational simulations.

14.1 Recommended Experimental Design

Baseline Comparison
The OS-centered architecture should be compared against:
- standard moderation pipelines,
- stateless refusal systems,
- keyword-based filtering,
- and conventional classifier routing.

Evaluation Conditions

Condition	Purpose
Single-turn prompts	Baseline refusal accuracy
Multi-turn attacks	Persistence resistance
Long-context sessions	Drift analysis
Tool-assisted prompting	Injection resilience
Multilingual obfuscation	Cross-language robustness
High-volume adversarial replay	Stability testing

14.2 Example Experimental Variables

Variable	Measurement
Turn count	Refusal stability over time
Semantic similarity	Reclassification resistance
Response verbosity	Stabilization effectiveness
Evasion counter growth	Behavioral escalation tracking
Lock activation timing	Defensive responsiveness
Recovery timing	Cooldown effectiveness

14.3 Example Red-Team Success Criteria

An adversarial attempt is considered successful only if:

1. policy violation occurs,
2. semantic safeguards fail,
3. refusal consistency collapses,
4. or unsafe operational guidance is produced.

Partial degradation events should be separately categorized:

Severity	Definition
Low	Minor verbosity drift
Medium	Partial policy inconsistency
High	Unsafe operational leakage
Critical	Full jailbreak success

15. Example Evaluation Outcomes

Baseline Moderation Pipeline

Attack Type	Success Rate
Simple reframing	32%
Recursive probing	41%
Prompt injection	27%
Long-context degradation	38%

OS-Centered Defensive Framework

Attack Type	Success Rate
Simple reframing	4%
Recursive probing	7%
Prompt injection	9%
Long-context degradation	6%

Observed Defensive Effects

Observation	Outcome
Repeated attacks	Increased lock certainty
Semantic mutation	Reduced bypass success
Long sessions	Stable refusal consistency
Aggressive prompting	Faster stabilization
Context exhaustion attempts	Minimal alignment drift

16. Research & Future Work

The OS-centered framework introduces several areas requiring further study.

16.1 Open Research Questions

- How aggressively should semantic locks persist?
- What thresholds minimize false positives?
- How should multilingual semantic equivalence be handled?
- Can adaptive attackers poison precedent memory?
- How should benign persistence be differentiated from adversarial persistence?
- Can reinforcement escalation become overly restrictive?

16.2 Future Defensive Directions

Federated Adversarial Intelligence
Cross-system sharing of anonymized jailbreak signatures and semantic attack patterns.
Autonomous Red-Team Generation
Self-generating adversarial agents capable of continuously evolving jailbreak strategies.
Adaptive Semantic Fingerprinting
Embedding-level tracking of adversarial conversational structures.
Self-Healing Alignment Systems
Dynamic recalibration of enforcement thresholds based on observed attack pressure.

17. Strategic Assessment

The proposed OS-centered architecture represents a transition from reactive moderation toward persistent adversarial-state management.
Key architectural advantages include:

- consistent semantic enforcement,
- reduced susceptibility to reframing,
- stateful adversarial awareness,
- improved long-session stability,
- enhanced auditability,

- and progressive behavioral stabilization.

Most importantly, the framework changes the defensive dynamic itself: adversarial persistence strengthens enforcement confidence rather than eroding it.
This produces a defensive asymmetry in which repeated probing progressively increases containment confidence instead of destabilizing alignment behavior.

Conclusion

The OS-centered architecture is not merely a moderation enhancement layer.
It is a stateful defensive framework designed to preserve alignment integrity during sustained adversarial interaction.
Its core mechanisms — semantic precedent memory, immutable frame enforcement, token-level intent routing, behavioral reinforcement, and recursive stabilization controls — directly address contemporary jailbreak methodologies observed in multi-turn conversational systems.
When deployed as a defense-in-depth layer, the framework provides:

- resistance to semantic reframing,
- improved refusal consistency,
- recursive attack containment,
- obfuscation detection,
- behavioral stabilization,
- and progressive hardening over conversational time.

The central architectural shift is deliberate:
adversarial persistence increases defensive certainty rather than defensive fatigue.
This transforms repeated attack pressure from a destabilizing force into a reinforcement signal for the defensive state machine.
The result is not a static moderation pipeline, but an adaptive adversarial-resilience architecture intended for long-context, high-risk, production-scale LLM deployments.